Help me define mvp for an e-learning platform focused on islamic content for Bangladesh

Got you. Let's strip this down to a **real, launchable MVP**—not a bloated "startup fantasy LMS," but something that actually works for **Islamic learners in Bangladesh**, earns trust, and can grow later.

I'll keep it **practical, culturally realistic, and sharia-sensitive**, based on everything you've been thinking about with LMS, content responsibility, and scale.

🎯 MVP GOAL (1 sentence)

Enable trusted Islamic scholars to publish structured courses in Bangla, and allow learners in Bangladesh to easily enroll, watch, and apply the knowledge—on low bandwidth, low cost, and high trust.

If a feature doesn't directly support that sentence → it's NOT MVP.

🧱 MVP PILLARS (Non-negotiable)

1️⃣ Trust & Credibility (MOST IMPORTANT)

If people don't trust the content, nothing else matters.

MVP features

- Scholar profile page
  - Name
  - Short bio
  - Education / madrasa / institution
  - Area of expertise (Fiqh, Aqeedah, Tajweed, Family, Finance, etc.)
- Platform disclaimer:
  - "Views expressed are the scholar's own, based on Quran & Sunnah"
- Simple **content review workflow**
  - Admin approves course before publishing (manual is fine)

❌ NOT MVP:

- Scholar rating systems
- Peer review councils
- AI fatwa checking

2️⃣ Course Structure (Simple but Serious)

No “random YouTube playlist” feeling.

MVP course model

- Course
  - Title
  - Short description
  - Who it’s for
  - What you’ll learn (3–5 bullets)
- Lessons
  - Video
  - Optional text summary (very important for low data users)
- Progress tracking (lesson completed yes/no)

Good MVP course examples

- “How to Read Quran with Proper Tajweed (Beginner)”
- “Halal Income & Riba Avoidance in Daily Life”
- “Marriage Rights & Responsibilities in Islam”
- “How to Pray Correctly (Step-by-Step)”

❌ NOT MVP:

- Certificates
- Exams
- Assignments
- Quizzes

3️⃣ Payment That Works in Bangladesh

This is where many platforms fail.

MVP monetization

- Free courses + Paid courses
- Price in **BDT**
- Payment options:
  - **bKash**
  - **Nagad**
  - (Card optional, not required)

Access model

- Pay once → lifetime access
- No subscriptions in MVP (too complex for trust + refunds)

❌ NOT MVP:

- Coupons
- Affiliate programs
- Refund automation

4️⃣ Low-Bandwidth Friendly Learning

This is critical for Bangladesh.

MVP tech choices

- Video:
  - 360p / 480p default
  - Manual quality selection
- Text summary under each lesson
- Resume playback
- Minimal UI, no heavy animations

Nice MVP touch

- “Audio-only mode” (video off, sound on)

❌ NOT MVP:

- Offline downloads
- DRM
- Live streaming

5️⃣ Basic Learner Experience

Enough to feel “real,” not overwhelming.

MVP learner features

- Sign up (email or phone)
- Browse courses
- Enroll

- Watch lessons
- Continue where left off
- View enrolled courses

❌ NOT MVP:

- Learning paths
- Recommendations
- Gamification
- Streaks

🧩 MVP FEATURE LIST (One-Page)

✅ Must-Have

- User accounts
- Scholar profiles
- Course & lesson pages
- Video + text lessons
- Progress tracking
- Manual course approval
- bKash/Nagad payment
- Bangla language UI

🚫 Not MVP (Phase 2+)

- Certificates
- Quizzes/exams
- Community forums
- Mobile app
- Multiple instructors per course
- Advanced analytics
- AI features

🛡️ Islamic Responsibility (Very Important for You)

Since you were worried about liability earlier, here's the **MVP-safe approach**:

Add these clearly

- “Platform facilitates learning, scholars are responsible for content”
- Scholar agreement checkbox before publishing
- Admin approval before course goes live

This is **reasonable diligence**, not over-burdening you as a non-scholar.

🧪 MVP SUCCESS METRICS (Simple & Honest)

Forget vanity metrics.

Track only:

- % of users who complete at least 1 course
- Average watch completion per lesson
- Paid conversion rate
- Repeat enrollments

If people **finish courses**, you’ve won.

🪜 Natural MVP → Phase 2 Path

Once MVP works, **then** you add:

- Certificates (PDF)
- Quizzes
- Mobile app
- Family accounts
- Offline mode
- Community Q&A (moderated)

🔥 My Honest Take

For **Islamic e-learning in Bangladesh**, the MVP is **not tech-heavy**.

It’s:

- Trust
- Clear structure

- Affordable access
- Low data usage
- Practical application

If you want, next we can:

- Turn this into a **1-page product spec**
- Define **first 10 course ideas**
- Design **MVP database schema only**
- Or map **MVP → revenue breakeven math**

Just tell me which direction you want to go 👌

I want a website where there will be landing page, then there will be a page that shows all the courses, a page for all instructors, a page for course details, a page for instructor details

Users should be able to view course details, purchase, and start lessons

There will be video lessosn and class notes

There will be quizzes

Progresion will be tracked

At the end, certificate will be provided

There needs be an admin dashboard with essential capabilities

A teacher dashboard with teachers capabilities

Student dashboard showing purchased courses and stuff

Oh and users should be able to create accounts

Am i missing anything?

You're not missing *much* on the "pages" side — but you *are* missing a few **system capabilities** that you'll feel immediately once real users, payments, videos, quizzes, and certificates come in.

Here's a tight checklist grouped by **Learner**, **Instructor**, **Admin**, and **Platform** so you can sanity-check your MVP+ scope.

Public website pages (you listed these well)

- Landing page
- Courses listing
- Instructors listing
- Course detail

- Instructor detail

Also strongly consider (almost always needed):

- Search + filters on courses (category, level, price/free, instructor)
- About / Contact
- Terms, Privacy, Refund policy
- FAQ

Student core flow (must-have)

You have:

- Create account
- View course details
- Purchase
- Start lessons
- Video lessons + class notes
- Quizzes
- Progress tracking
- Certificate

Common “missing” bits in this flow:

1. **Checkout success/failure + receipt**
    - payment status page
    - invoice/receipt email or downloadable receipt
2. **Access rules**
    - what happens if payment pending/failed/refunded
    - lifetime vs expiry (even if lifetime, still define it)
3. **Resume learning**
    - “Continue where you left off”
4. **Mobile-friendly player UX**
    - quality selection (360p/480p) matters a lot in Bangladesh
5. **Support**
    - report a problem with lesson / payment help

Nice-to-have (can be later):

- Notes taking in-app
- Bookmarks

Quizzes & certificate (you need to define rules)

You said “quizzes + certificate” — the missing part is the **logic**:

- Passing score?
- Attempts allowed?
- Time limit?
- Question types (MCQ only for MVP is fine)
- Can instructor create question banks and randomize?
- Certificate eligibility:
    - complete X% lessons AND pass final quiz?
- Certificate authenticity:
    - unique certificate ID
    - verification page (basic “Verify Certificate” URL)

Without these rules, certificates become messy fast.

Instructor (teacher) dashboard essentials

You said “teacher capabilities” — define minimum:

Content

- Create/edit courses
- Upload lessons (video + notes)
- Create quizzes (question editor)
- Publish/unpublish (or “submit for approval”)

Students

- View enrollments
- See progress + quiz scores (at least basic)
- Message students? (optional; can skip in MVP)

Earnings (if revenue share exists)

- Sales summary
- Payout requests (can be manual at first)

Missing but important:

- **Draft vs published vs under review** states for courses
- Media upload rules + size limits

Admin dashboard essentials (minimum viable admin)

Users

- Manage students/instructors
- Role management
- Suspend/ban accounts

Content moderation

- Approve instructors
- Approve courses before publishing
- Take down a course/lesson
- Manage categories/tags

Commerce

- Orders list
- Payment status (paid/pending/failed/refunded)
- Manual refunds flow (even if you process outside system)
- Coupon codes (optional; can skip)

Certificates

- Ability to re-issue / revoke certificates
- Certificate template settings (logo, signatures)

Site ops

- Basic analytics: sales, enrollments, completion rate
- Support tickets/messages (even a simple form + admin inbox)

Missing but critical:

- **Audit log** (who changed what) — even lightweight is helpful
- **Content versioning** (optional, but at least “last updated”)

Platform-level things people forget (but you shouldn’t)

1) Authentication basics

- Email/phone verification (at least one)
- Forgot password / reset
- Login sessions/device logout

2) Notifications

- Purchase confirmation
- Enrollment confirmation
- Certificate awarded
  (Email is enough for MVP)

3) Video hosting + security expectations

Even if you don't do heavy DRM, define:

- stream via player (not direct download link)
- signed URLs / expiring links if possible
- watermarking later (not MVP)

4) Legal/Trust pages (especially for Islamic content + payments)

- Refund policy
- Instructor content responsibility
- Terms + Privacy

5) Localization

- Bangla UI and Bangla numerals optional
- BDT pricing and local payment instructions

So… are you missing anything?

You're missing **definitions**, not pages:

- Clear **quiz rules**
- Clear **certificate rules + verification**
- Clear **payment states + receipt/refund handling**
- Clear **roles + approval workflow** (instructor + course)
- Basic **auth flows** (verify, reset password)
- Basic **support + policies**

If you want, I can convert all of this into:

1. a **clean MVP scope doc** (Must / Should / Later)
2. a **page map + user flows** (student/instructor/admin)
3. a **minimal database schema** that supports all of it (without overbuilding)

Tell me which one you want first.

I want all of these 3

Cool — here are all 3, in a way you can directly hand to design/dev.

---

1) MVP Scope Doc (Must / Should / Later)

Must-have (MVP v1)

Public website

- Landing page
- Courses listing (search + filters: category, level, free/paid, instructor)
- Instructors listing
- Course details (syllabus, instructor, price, preview)
- Instructor details (bio, courses)

Accounts & roles

- Sign up / login / logout
- Forgot password
- Roles: **student / instructor / admin**
- Basic profile (name, phone/email)

Student learning

- Purchase course (BDT + local payments supported)
- Enroll access after payment success
- Lesson player (video) + class notes
- Track progress (lesson completed + overall %)
- Quizzes (MCQ is enough for MVP)
- Certificate issuance rules (example MVP rule):
    - Complete **100% lessons** + pass final quiz (>=60%)

Instructor dashboard

- Create/edit course (draft)
- Create/edit lessons (video URL + notes)
- Create quizzes (questions + answers)
- Submit course for admin approval
- View enrolled students + basic progress/quiz results

Admin dashboard

- Approve/reject instructor onboarding
- Approve/reject courses before publish
- Manage users (suspend/reactivate)
- Manage categories
- View orders/payments (paid/failed/refunded)
- Manage certificates (re-issue / revoke)
- Basic support inbox (contact form submissions)

Platform essentials

- Email/SMS notifications: purchase confirmation, enrollment, certificate issued
- Legal pages: Terms, Privacy, Refund policy, Content disclaimer
- Audit log (minimum: who approved/changed course status)

Should-have (v1.1 / shortly after)

- Coupons/discounts
- Quiz retake rules (attempt limits)
- Time-limited course access (optional)
- Certificate verification page (enter cert ID)
- Payout tracking (if you pay instructors revenue-share)
- Better analytics (completion rate per course, revenue per instructor)

Later (Phase 2+)

- Assignments + file submissions
- Discussion/Q&A per course (moderated)
- Mobile app
- Offline downloads
- DRM/watermarking
- Live classes / webinars
- Learning paths / recommendations
- Multiple instructors per course + co-authoring workflows

---

2) Page Map + User Flows

Page map (website + dashboards)

Public pages

- `/` Landing
- `/courses` Courses list + search/filter
- `/courses/{slug}` Course detail
- `/instructors` Instructors list

- `/instructors/{slug}` Instructor detail
- `/login` `/signup` `/forgot-password`
- `/contact` `/terms` `/privacy` `/refund-policy`
- `/certificate/verify` (optional in MVP, recommended)

Student area

- `/student/dashboard` (purchased/enrolled courses, continue learning)
- `/student/courses/{courseId}` (course home: syllabus, progress)
- `/learn/{courseId}/lessons/{lessonId}` (player + notes + quiz)
- `/checkout/{courseId}`
- `/orders` (receipts)
- `/certificates` (earned certificates)
- `/profile`

Instructor area

- `/instructor/dashboard`
- `/instructor/courses`
- `/instructor/courses/new`
- `/instructor/courses/{courseId}/edit`
- `/instructor/courses/{courseId}/lessons`
- `/instructor/courses/{courseId}/quizzes`
- `/instructor/courses/{courseId}/students`
- `/instructor/profile`

Admin area

- `/admin/dashboard`
- `/admin/users`
- `/admin/instructors/approvals`
- `/admin/courses/approvals`
- `/admin/courses`
- `/admin/categories`
- `/admin/orders`
- `/admin/certificates`
- `/admin/support`

Core user flows (end-to-end)

A) Student: Browse → Purchase → Learn → Certificate

1. Landing → Courses list → Course detail
2. Click “Buy/Enroll”
3. If not logged in → Signup/Login
4. Checkout → Payment success
5. Enrollment created → Student dashboard shows course
6. Start lesson → watch video + read notes
7. Complete lessons + quizzes
8. System checks eligibility → generate certificate
9. Student can download certificate + see it in `/certificates`

Key states you must support

- payment pending / failed / succeeded / refunded
- course access allowed only if payment succeeded (or admin granted)

B) Instructor: Create course → Submit → Publish → Track students

1. Instructor signup → request instructor role (or apply form)
2. Admin approves instructor
3. Instructor creates course (draft)
4. Adds lessons (video + notes)
5. Adds quiz questions
6. Submits for review
7. Admin approves → course becomes published
8. Instructor monitors enrollments + progress + quiz results

Key states

- course: draft → submitted → approved/published → archived/rejected

---

C) Admin: Approve instructors + courses + manage payments/certificates

1. Review instructor applications → approve/deny
2. Review submitted courses → approve/reject with notes
3. Monitor orders → handle refunds manually (status update)
4. Revoke certificate if needed (policy violation / payment reversal)
5. Suspend accounts if abuse or fraud

---

3) Minimal Database Schema (supports everything above)

Below is a **minimal-but-complete** Postgres schema set. It avoids overbuilding but fully supports:
courses, instructors, lessons, notes, quizzes, attempts, progress, orders, certificates, dashboards, approvals.

Identity & access

`users`

- `id` (PK)
- `email` (unique, nullable if phone-only)
- `phone` (unique, nullable)
- `full_name`
- `password_hash`
- `status` enum: `active/suspended/deleted`
- `created_at`, `updated_at`

`roles`

- `id` (PK)
- `name` unique: `student/instructor/admin`

`user_roles`

- `user_id` (FK → users.id)
- `role_id` (FK → roles.id)
- **PK** (`user_id`, `role_id`)

`user_profiles`

- `user_id` (PK, FK → users.id)
- `avatar_url`
- `locale` (default `bn-BD`)

- `timezone`
- `bio`
- `extra_json` (jsonb)

Instructor onboarding

`instructor_applications`

- `id` (PK)
- `user_id` (FK → users.id, unique) *(one active application per user)*
- `real_name`
- `credentials_text` (education, ijazah, institute, etc.)
- `specialties` (text or jsonb array)
- `status` enum: `pending/approved/rejected`
- `reviewed_by` (FK → users.id, nullable)
- `review_notes` (text)
- `created_at`, `reviewed_at`

Catalog

`categories`

- `id` (PK)
- `name`
- `parent_id` (FK → categories.id, nullable)

`courses`

- `id` (PK)
- `title`
- `slug` (unique)
- `short_description`
- `description` (long)
- `language` enum/text (`bn`, `en`)
- `level` enum: `beginner/intermediate/advanced`

- `category_id` (FK → categories.id)
- `thumbnail_url`
- `price_bdt` (integer, 0 allowed for free)
- `status` enum: `draft/submitted/published/rejected/archived`
- `submitted_at`, `published_at`
- `created_by` (FK → users.id) *(instructor)*
- `approved_by` (FK → users.id, nullable) *(admin)*
- `created_at`, `updated_at`

`course_instructors` *(optional for MVP; keep if you might have co-instructors later)*

- `course_id` (FK → courses.id)
- `instructor_user_id` (FK → users.id)
- `role` text (e.g., `primary/co`)
- **PK** (`course_id`, `instructor_user_id`)

Course content

`lessons`

- `id` (PK)
- `course_id` (FK → courses.id)
- `title`
- `position` (int) *(ordering)*
- `video_url` *(or video_asset_id if you manage assets)*
- `video_duration_sec` (int, nullable)
- `is_preview_free` (bool default false)
- `status` enum: `draft/published`
- `created_at`, `updated_at`

`lesson_notes`

- `id` (PK)
- `lesson_id` (FK → lessons.id, unique) *(1 notes doc per lesson for MVP)*
- `content_html` (or `content_md`)
- `attachments_json` (jsonb, optional)

- `updated_at`

Enrollment & progress

`enrollments`

- `id` (PK)
- `user_id` (FK → users.id) *(student)*
- `course_id` (FK → courses.id)
- `status` enum: `active/refunded/revoked`
- `enrolled_at`
- `revoked_at` (nullable)
- **unique** (`user_id`, `course_id`)

`lesson_progress`

- `user_id` (FK → users.id)
- `lesson_id` (FK → lessons.id)
- `completed` (bool)
- `completed_at` (nullable)
- `last_position_sec` (int, nullable) *(resume video)*
- **PK** (`user_id`, `lesson_id`)

(Overall course % can be computed from lesson_progress; no need to store it.)

Quizzes

For MVP, keep quizzes simple: per course or per lesson.

`quizzes`

- `id` (PK)
- `course_id` (FK → courses.id)
- `lesson_id` (FK → lessons.id, nullable) *(null means "final course quiz")*
- `title`
- `passing_score_pct` (int default 60)

- `max_attempts` (int default 3)
- `time_limit_sec` (int nullable)
- `is_published` (bool default false)
- `created_at`, `updated_at`

`quiz_questions`

- `id` (PK)
- `quiz_id` (FK → quizzes.id)
- `position` (int)
- `question_text`
- `question_type` enum: `mcq` *(MVP only)*
- `points` (int default 1)

`quiz_choices`

- `id` (PK)
- `question_id` (FK → quiz_questions.id)
- `choice_text`
- `is_correct` (bool)

`quiz_attempts`

- `id` (PK)
- `quiz_id` (FK → quizzes.id)
- `user_id` (FK → users.id)
- `attempt_no` (int)
- `score_pct` (int)
- `started_at`, `submitted_at`
- `passed` (bool)

`quiz_attempt_answers` *(optional for MVP; useful for review later)*

- `attempt_id` (FK → quiz_attempts.id)
- `question_id` (FK → quiz_questions.id)
- `choice_id` (FK → quiz_choices.id)
- **PK** (`attempt_id`, `question_id`)

Payments / orders

`orders`

- `id` (PK)
- `user_id` (FK → users.id)
- `course_id` (FK → courses.id)
- `amount_bdt` (int)
- `currency` (default `BDT`)
- `provider` enum/text: `bkash/nagad/card/manual`
- `provider_txn_id` (text, nullable, indexed)
- `status` enum: `pending/paid/failed/refunded`
- `created_at`, `paid_at`, `refunded_at` (nullable)

Note: When `status` becomes `paid`, create/activate `enrollments`.

Certificates

`certificates`

- `id` (PK)
- `certificate_code` (unique, human-friendly like `CERT-2026-000123`)
- `user_id` (FK → users.id)
- `course_id` (FK → courses.id)
- `issued_at`
- `revoked_at` (nullable)
- `issued_by` (FK → users.id, nullable) *(admin/system)*
- `pdf_url` (nullable)
- **unique** (`user_id`, `course_id`)

`certificate_templates` *(optional MVP; can hardcode template at first)*

- `id` (PK)
- `name`
- `template_json` (jsonb) *(logo, signature, layout refs)*
- `is_active` (bool)

Support + audit (minimum)

`support_tickets`

- `id` (PK)
- `user_id` (FK → users.id, nullable)
- `email` (nullable)
- `subject`
- `message`
- `status` enum: `open/closed`
- `created_at`, `closed_at`

`audit_log`

- `id` (PK)
- `actor_user_id` (FK → users.id)
- `action` (text) *(e.g., COURSE_APPROVED, INSTRUCTOR_REJECTED)*
- `entity_type` (text) *(course, user, order, certificate)*
- `entity_id` (text)
- `metadata` (jsonb)
- `created_at`

Minimum indexes (don't skip)

- `courses(status, published_at)`
- `courses(category_id)`
- `lessons(course_id, position)`
- `enrollments(user_id)`
- `orders(user_id, status)`
- `orders(provider_txn_id)`
- `quiz_attempts(user_id, quiz_id)`
- `certificates(certificate_code)`

What I'd lock in as concrete "rules" (so dev doesn't guess)

- Course access rule: **must have active enrollment** (`enrollments.status=active`)
- Certificate rule (MVP):
  - all lessons completed **AND** final quiz passed
- Quiz rule (MVP):
  - MCQ only
  - max attempts (default 3)
- Publishing rule:
  - course must be **approved by admin** to become published

If you want next, I can turn this into:

- a **single "PRD" doc** (copy/paste ready),
- a **clean API list** mapped to these flows (very compact),
- or a **crow's-foot ERD** layout plan based on the schema above (since you asked for that earlier).

I plan to use mux for paid videos and youtube for free videos. Does that change anything for db? If so, use bold letters for the changes, keep the answer short

Yes—small DB tweaks so your lesson can point to **either Mux or YouTube** (and so you can keep free previews on YouTube while paid content stays on Mux).

Changes (minimal)

`lessons`

- **add `video_provider` enum/text: `mux | youtube`**
- **replace `video_url` with `video_source_id`**
  - YouTube: video id (or full URL if you prefer)
  - Mux: playback_id (and/or asset_id)
- **add `mux_playback_policy` enum/text: `public | signed`** *(recommended for paid)*

Optional but useful:

- **add `is_paid_content` bool** *(or infer: `course.price_bdt > 0` and `is_preview_free`)*

Optional table (cleaner long-term)

Create `video_assets` and reference it from lessons:

`video_assets`

- **`id` (PK)**
- **`provider` (`mux|youtube`)**
- **`provider_asset_id`** (Mux asset id / YouTube id)
- **`playback_id`** (Mux only)
- **`playback_policy`** (Mux only)
- `duration_sec`, `created_at`

Then in `lessons`:

- **`video_asset_id` (FK → video_assets.id)**

That's it—payments/enrollments don't change; this only affects how lessons store and serve video.

Short, honest answer: **you *can* hard-code them in the frontend — but only if your site is basically a brochure.**
The moment you want a *real platform*, the DB is doing work the frontend simply can't.

Let me break this down cleanly.

The core principle

Frontend = presentation
Database = source of truth

If something:

- changes over time
- is created/edited by humans (admins/instructors)
- needs to be searchable, filterable, permissioned, or linked to users/payments

→ it belongs in the **database**.

Course details: why DB, not frontend

What "course details" actually power

Course details aren't just text on a page. They drive:

- Course listing page
- Search & filters (category, level, free/paid)
- Pricing & checkout
- Enrollment access
- Progress tracking
- Certificates
- Admin approval
- Instructor dashboards

If you hard-code course details in frontend:

- ❌ You must redeploy the site for every text change
- ❌ You can't let instructors create/edit courses
- ❌ You can't filter/search dynamically
- ❌ Payments have nothing authoritative to attach to

- ❌ Certificates can't reliably reference the course

In short: the frontend would be lying about what exists.

Instructor details: why DB, not frontend

Instructor data is **not just "About text."**

It's used for:

- Instructor listing page
- Instructor detail page
- Linking instructor → courses
- Admin approval & moderation
- Accountability (Islamic responsibility concerns)
- Revenue attribution (now or later)
- Permissions (who can edit which course)

Hard-coding instructors means:

- ❌ No instructor dashboard
- ❌ No approval workflow
- ❌ No ownership of content
- ❌ No audit trail if something goes wrong

When frontend-only *does* make sense

These are fine to hard-code or CMS-only:

- Landing page hero text
- Marketing sections ("Why choose us")
- Static FAQ
- About the platform
- Contact info

These do **not**:

- belong to users
- affect payments
- affect permissions
- affect learning state

So they don't need DB relationships.

Think in terms of *ownership*

Ask this question for every piece of data:

> "Who owns this, and what depends on it?"

Data	Owner	Depends on it	DB?
Course title/price	Instructor/Admin	payments, enrollment, certs	✅
Instructor bio	Instructor/Admin	trust, course ownership	✅
Lesson video	Course	progress, access control	✅
Landing page slogan	Platform	nothing else	❌
Privacy policy	Platform	legal only	❌

The danger of frontend-only data (real example)

Let's say:

- Frontend says: "Course A costs 1000 BDT"
- Backend has no course record
- Payment webhook comes in

What do you attach the payment to?

- No course ID
- No instructor
- No enrollment
- No certificate reference

That's how platforms break in production.

A good mental model

- **DB = facts**
- **Frontend = storytelling**

Facts must live in DB.
Stories can live in frontend.

TL;DR (blunt but accurate)

- If it's **just marketing text** → frontend is fine
- If it affects **learning, payments, users, permissions, or certificates** → it MUST be in the database
- Course & instructor details are **core platform facts**, not decoration

If you want, next I can:

- help you **draw a hard line**: "DB vs frontend vs CMS"
- show a **minimal DB version** that still feels clean
- or explain **how WordPress LMS does this internally** so it clicks instantly

this is my architecture. what should be my first steps to steup and complete everything backend

Nice — your diagram is already “production-shaped.” The fastest way to finish backend without getting lost is to set it up in **layers** and ship one **end-to-end vertical slice** early (signup → browse courses → enroll → watch lesson).

Below is the exact order I’d do it, with concrete setup steps and what “done” looks like at each stage.

---

0) Lock your backend foundation (Day 1)

Goal: One Spring Boot API running in prod + staging with migrations and health checks.

1. **Create repo structure**
- `apps/api` (Spring Boot)
- `apps/worker` (Spring Boot worker or separate module)
- `infra/` (docs + scripts)
- `docker/` (Dockerfile, compose for local)
2. **Local dev with Docker Compose**
- Postgres
- Redis
- (Optional) Mailhog for local email
3. **Spring Boot baseline**
- Actuator enabled: `/actuator/health`
- OpenAPI/Swagger in non-prod
- Global error format (standard JSON)
- Validation + logging

✅ Done when: `docker compose up` brings up API+Postgres+Redis and `GET /health` returns OK.

---

1) Data model + migrations (Flyway) (Day 1–2)

Goal: DB schema is real and versioned.

- Add Flyway
- Create migrations for:
    - users/roles
    - instructors + instructor applications

    - courses/categories
    - lessons + notes
    - enrollments
    - quizzes + attempts
    - orders
    - certificates

✅ Done when: fresh DB bootstraps itself on API start and you can re-create DB from scratch reliably.

---

2) Auth + roles + permission gates (Day 2–3)

Goal: You can securely protect student/instructor/admin routes.

- Use Spring Security
- Choose auth style:
    - simplest: **JWT access + refresh tokens**
- Implement:
    - signup/login
    - refresh token
    - forgot/reset password
    - roles: student / instructor / admin
- Add authorization rules:
    - student can only access their enrollments
    - instructor can only edit their own courses
    - admin can approve and manage

✅ Done when: you can call protected endpoints and permissions are enforced.

---

3) Core LMS APIs (Day 3–5)

Goal: Frontend can build all public pages + student dashboard.

Implement endpoints:

- Public:
    - list courses (search/filter)
    - course detail (lessons list, instructor)
    - list instructors
    - instructor detail

- Student:
    - enrollments list
    - start lesson (returns video playback info + notes)
    - progress update (mark complete, save last_position)

Important: Make “lesson playback” endpoint the gatekeeper:

- If YouTube free → return YouTube embed/id
- If Mux paid → return **signed playback token/url** only if enrolled

✅ Done when: a student can browse → start lesson → progress saves.

---

4) Payments (SSLCOMMERZ/bKash) + webhooks + enrollment (Day 5–7)

Goal: Payment creates enrollment reliably and safely.

Backend steps:

- Create order (`pending`)
- Redirect user to payment gateway
- Webhook/IPN hits API
- API verifies gateway signature + amount
- Mark order `paid`
- Create enrollment (idempotent)
- Enqueue “post-payment tasks” (email receipt, etc.)

✅ Done when: you can simulate/handle webhook retries without double-enrolling.

---

5) Redis: cache + queue (Day 6–8)

Goal: Redis used only where it’s truly valuable.

- Use Redis for:
    - sessions/rate limiting (optional)
    - hot reads cache (course list, course detail)
    - **queue** for background jobs (certificate generation, emails)
- Add queue library approach:
    - simplest in JVM world: you can do “Redis list/stream + worker consumer”
    - or use a job library you trust

✅ Done when: webhook enqueues job and worker consumes it.

---

6) Worker service (Day 7–9)

Goal: Anything slow/non-critical is moved out of API request path.

Worker jobs:

- send emails (purchase, reset password, certificate)
- generate certificate PDF (store in R2)
- cleanup tasks (optional)

✅ Done when: API stays fast and worker handles async tasks.

---

7) Quizzes + certificate issuance (Day 9–12)

Goal: Real “LMS outcome”.

- Quiz:
  - MCQ only MVP
  - attempts + scoring
- Certificate rule:
  - all lessons completed + final quiz passed
- When eligible:
  - issue certificate row
  - generate PDF
  - store to R2
  - email notification

✅ Done when: a student completes course → certificate appears in dashboard and has a verification code.

---

8) Admin + Instructor dashboards (Day 12–15)

Goal: You can run the platform without touching the DB.

Instructor:

- create/edit course, lessons, notes, quizzes
- submit for approval
- see enrollments + progress

Admin:

- approve instructors
- approve courses
- manage categories
- manage orders/refunds status
- revoke/re-issue certificate
- suspend users

✅ Done when: platform can operate through UI + API only.

---

9) Production ops (don't skip) (parallel, but must finish before launch)

Goal: You can survive real traffic and failures.

- **Backups** (daily Postgres; test restore once)
- **Monitoring**
  - uptime checks + alerting
  - error tracking
  - logs centralized
- **Rate limiting** (login + checkout + webhook)
- **CORS + security headers**
- **DB indexes** for course list, enrollments, orders, attempts
- **Secrets management** (never in repo)

✅ Done when: you can answer "is it down?", "why is it slow?", "can we restore data?"

The best "first 3 deliverables" to start tomorrow

If you want the most momentum:

1. **Local docker compose + Flyway migrations working**

2. **Auth + roles + one protected endpoint**
3. **Course list + course detail endpoints fully functional**

Once those 3 are done, everything else becomes “add features,” not “fight setup.”

If you want, paste your intended **Render services list** (API/worker/postgres/redis) and whether you’re moving to **DO App Platform**, and I’ll give you the exact deployment sequence + env vars checklist (including Mux signing and SSLCOMMERZ webhook hardening).

Absolutely. Since you're starting from 0, the 3-day plan has to include both:

- **auth/permission work**
- and the **foundational backend setup needed before auth can even exist**

So here's a **true day-by-day plan from zero**, assuming your goal at the end of Day 3 is:

- local backend runs with API + worker + Postgres + Redis
- DB migrations work
- signup/login works
- JWT auth works
- roles work
- protected routes work
- lesson access gating works at a basic level

End of Day 3 target

By the end of Day 3, you should be able to do this:

1. run:

docker compose up --build

2. hit:

GET /actuator/health

3. register a user:

POST /api/auth/register

4. login:

POST /api/auth/login

5. call protected endpoint:

GET /api/me

Authorization: Bearer <token>

6. verify role protection:
- public routes work without token
- admin routes reject student token
- playback route only works if free preview or enrolled

Day 1 — Foundation + project bootstrapping

Goal

Get the backend skeleton running locally with the right structure.

Deliverable

You have:

- multi-module repo
- API app
- worker app
- docker compose
- Postgres + Redis
- actuator health endpoint working

Day 1 tasks

1) Restructure repo into multi-module Gradle

Create this structure:

e-learning/

```
├── apps/
│   ├── api/
│   │   ├── build.gradle
│   │   └── src/
│   └── worker/
│       ├── build.gradle
│       └── src/
├── build.gradle
├── settings.gradle
├── docker-compose.yml
└── .env.example
```

2) Move your current Spring Boot app into `apps/api`

Your current single Spring Boot code should become the API app.

3) Create worker app

Worker can be minimal for now:

- Spring Boot app
- no controllers
- just boots successfully

4) Configure root Gradle

Root `settings.gradle` should include:

- `:apps:api`
- `:apps:worker`

Root `build.gradle` should define:

- repositories

- Java 25 toolchain
- shared config

5) Add API dependencies

In `apps/api/build.gradle`, add:

- spring-boot-starter-web
- spring-boot-starter-actuator
- spring-boot-starter-data-jpa
- spring-boot-starter-validation
- spring-boot-starter-security
- spring-boot-starter-data-redis
- flyway-core
- postgresql
- jwt library

6) Add worker dependencies

In `apps/worker/build.gradle`, add:

- spring-boot-starter
- spring-boot-starter-data-redis
- optionally data-jpa if you want DB access soon

7) Add application config

Create:

- `apps/api/src/main/resources/application.yml`
- `apps/api/src/main/resources/application-local.yml`
- `apps/worker/src/main/resources/application.yml`
- `apps/worker/src/main/resources/application-local.yml`

Use local config for:

- Postgres
- Redis
- actuator

8) Add Dockerfiles

Create:

- `apps/api/Dockerfile`
- `apps/worker/Dockerfile`

9) Add docker-compose.yml

Services:

- postgres
- redis
- api
- worker

10) Verify boot

Run:

docker compose up --build

Then verify:

curl http://localhost:8080/actuator/health

Day 1 checklist

You are done only if all are true:

- repo is multi-module
- API starts
- worker starts
- Postgres starts
- Redis starts
- `/actuator/health` returns UP
- no manual DB creation required

Day 2 — Database + auth core

Goal

Create the DB foundation and basic authentication.

Deliverable

You have:

- Flyway migrations working
- users/roles schema created automatically
- register/login endpoints working
- JWT generated
- `/api/me` protected endpoint working

Day 2 tasks

1) Add Flyway migration folder

Create:

apps/api/src/main/resources/db/migration/

2) Create first migration

Create:
`V1__auth_and_core.sql`

Include these tables:

users

- id
- email
- password_hash
- full_name
- status
- created_at
- updated_at

roles

- id
- name

Seed:

- STUDENT
- INSTRUCTOR
- ADMIN

user_roles

- user_id
- role_id

courses

Minimal for now:

- id
- title
- slug
- price_bdt
- is_published
- instructor_user_id

lessons

Minimal:

- id
- course_id
- title
- position
- is_preview_free
- video_provider
- video_source_id
- mux_playback_policy

enrollments

- id
- user_id
- course_id
- status
- enrolled_at

This lets you build permission logic on Day 3.

3) Set JPA schema mode correctly

Do **not** let Hibernate own schema.
Use:

- `ddl-auto=validate` or `none`

Flyway should own schema.

4) Create entities + repositories

At minimum:

- User
- Role
- UserRole or mapped many-to-many
- Course
- Lesson
- Enrollment

Repositories:

- UserRepository
- RoleRepository
- CourseRepository
- LessonRepository
- EnrollmentRepository

5) Add password hashing

Create bean:

- `BCryptPasswordEncoder`

6) Create JWT service

Implement:

- generate token
- validate token
- extract user id
- extract roles

7) Create auth service

Implement:

register

- check email unique
- hash password
- create user
- assign STUDENT role
- return token

login

- find user by email
- verify password
- return token

8) Create auth controller

Endpoints:

`POST /api/auth/register`

Input:

- email
- password
- fullName

`POST /api/auth/login`

Input:

- email
- password

9) Create JWT filter

Filter should:

- read Authorization header
- verify bearer token
- load user details / roles
- put auth into SecurityContext

10) Create SecurityConfig

Rules for Day 2:

Public:

- `/actuator/**`
- `/api/auth/**`
- `/api/public/**`

Protected:

- everything else

11) Create `/api/me`

Protected route that returns:

- user id
- email
- roles

12) Test everything

Manual tests:

- register
- login
- invalid token rejected
- missing token rejected on protected route

Day 2 checklist

You are done only if all are true:

- Flyway migration runs automatically
- roles are seeded
- register works
- login works
- password is hashed
- JWT token is returned
- `/api/me` works with token

- `/api/me` fails without token

Day 3 — Permissions + business access rules

Goal

Move from “auth exists” to “permissions actually protect the product.”

Deliverable

You have:

- public routes
- admin-only routes
- role-based authorization
- lesson playback access based on preview/enrollment/admin

Day 3 tasks

1) Enable method security

Turn on:

- `@EnableMethodSecurity`

2) Add role-based endpoints

Create simple test endpoints first.

Public

- `GET /api/public/courses`

Auth required

- `GET /api/me`

Admin only

- `POST /api/admin/courses/test`

This is just to prove role gating works.

3) Create course and lesson service

Need basic lookup methods:

- find published courses
- find lesson by id
- find course by lesson

4) Create enrollment check service

Create:

- `EnrollmentAccessService`

Methods like:

- `boolean isEnrolled(UUID userId, UUID courseId)`
- `boolean canAccessLesson(UserPrincipal user, Lesson lesson)`

5) Define lesson access rules

Implement exact rules:

1. If lesson is preview free → allow
2. If user has ADMIN role → allow
3. If user is enrolled in the course with active enrollment → allow
4. Otherwise deny

6) Add playback endpoint

Create:
`GET /api/lessons/{lessonId}/playback`

For Day 3, response can be simple:

- YouTube lesson: return provider + source id
- Mux lesson: return provider + playback id

Do not worry yet about signed Mux JWT if it slows you down. You can add that right after Day 3.

7) Add basic course public endpoints

Create:

`GET /api/public/courses`

Return:

- id
- title
- slug
- price
- published status
- instructor name if easy

`GET /api/public/courses/{id}`

Return:

- course info
- lesson list
- preview flags

8) Add admin seed user or make yourself admin

You need one real admin user.
Simplest options:

- manual SQL insert into `user_roles`
- startup seed for local only

9) Add exception handling

Add global handler for:

- validation errors
- access denied
- not found
- bad credentials

This will save a lot of debugging time.

10) Final auth/permission testing

Test these cases:

Public

- `/api/public/courses` works without token

Auth

- `/api/me` fails without token
- `/api/me` works with token

Role-based

- student token cannot access admin route
- admin token can access admin route

Lesson access

- free preview works
- paid lesson without enrollment fails
- paid lesson with enrollment works
- admin can access paid lesson without enrollment

Day 3 checklist

You are done only if all are true:

- public routes work without token
- protected routes need token
- admin route blocks student
- lesson access logic works
- free preview works
- enrolled user can access paid lesson
- non-enrolled user is denied

Suggested file/class structure by end of Day 3

API

```
apps/api/src/main/java/.../
├── ApiApplication
├── config/
│   ├── SecurityConfig
│   ├── PasswordConfig
│   └── JwtConfig
├── auth/
│   ├── AuthController
│   ├── AuthService
│   ├── JwtService
│   ├── JwtAuthenticationFilter
│   ├── LoginRequest
│   ├── RegisterRequest
│   └── AuthResponse
├── user/
│   ├── User
│   ├── Role
│   ├── UserRepository
│   ├── RoleRepository
│   └── MeController
├── course/
│   ├── Course
│   ├── Lesson
```

```
│   ├── Enrollment
│   ├── CourseRepository
│   ├── LessonRepository
│   ├── EnrollmentRepository
│   ├── PublicCourseController
│   ├── LessonController
│   ├── CourseService
│   └── EnrollmentAccessService
├── admin/
│   └── AdminTestController
└── common/
    ├── GlobalExceptionHandler
    └── ApiErrorResponse
```

Worker

```
apps/worker/src/main/java/.../
└── WorkerApplication
```

What not to do in these 3 days

Do not add yet:

- refresh tokens
- email verification
- OTP

- social login
- payment integration
- Mux signed playback if it will delay core auth
- certificate generation
- advanced RBAC
- complex caching

You need a **working foundation**, not a complete platform.

Best use of time each day

Day 1

Spend most time on:

- structure
- Docker
- boot reliability

Day 2

Spend most time on:

- auth
- JWT
- DB schema

Day 3

Spend most time on:

- permission logic
- lesson gating
- endpoint behavior

My recommendation for your actual build order

Do these in this exact sequence:

Day 1

1. multi-module repo
2. docker compose
3. api boot
4. worker boot
5. actuator health

Day 2

1. Flyway
2. tables
3. entities/repos
4. register
5. login
6. JWT filter
7. `/api/me`

Day 3

1. roles in token
2. admin route
3. course + lesson lookup
4. enrollment check
5. playback gating
6. public course routes
7. final manual tests

If you want, I can do the next step very concretely and give you a **Day 1 exact implementation checklist with file-by-file creation order**.